RhoForge:
A Semantics-Preserving GPU Reaction Compiler for CeTTa

Oruži Zarathustra Goertzel

August 2026 — design specification

Abstract

This document specifies a GPU backend for the reflective rho-calculus engine in CeTTa. The objective is not to place the existing pointer-based interpreter on a device. It is to compile canonical rho processes into a flat reaction representation in which channel matching is a segmented join, endpoint ownership is an occurrence-indexed claim, communication is a verified rewrite, and residual construction is parallel compaction. Three execution contracts are kept distinct: one-path scheduled execution, exact successor-frontier enumeration, and serializable waves of independent communications. A static-graph specialization maps event-driven neural networks and conserved causal-credit learning to fixed-width device records, allowing inference and local learning without an autograd graph or a PyTorch runtime. Correctness is fail-closed: every optimization must agree with the canonical CPU frontier or emit a replayable reduction certificate. The design therefore treats the rho semantics as the specification and the GPU layout as a refinement, not as a replacement language.

Status. This is an implementation specification, not a performance report. It is grounded in the current CeTTa rho engine and its test surfaces, but no GPU backend is claimed to exist yet. "Must", "shall", and acceptance gates below are normative.

1 Objective and boundary

The target is a direct path

rho/CeTTa source $\longrightarrow$ canonical reaction IR $\longrightarrow$ GPU execution $\longrightarrow$ rho residual or frontier.

For neural workloads, the same path shall carry locally generated activity, eligibility receipts, success signals, and conserved credit. There is no tensor autograd graph, backward tape, optimizer object, or Python runtime in the required execution path.

The backend is not required to accelerate every rho process. A single small, highly reflective process may be faster on the CPU. The GPU target is the regime with one or more of the following:

- many endpoints or many independent process instances;
- stable topology with changing messages;
- wide channel-local reaction frontiers;
- repeated event-driven neural episodes; or
- cost-accounted waves with substantial independent work.

Requirement 1.1 (Semantic authority)**.** *The existing strict-core and cost-profile semantics remain authoritative. No kernel may invent a reaction, omit an enabled reaction in an exact mode, or identify two occurrences merely because their payloads are equal.*

2 Verified baseline in the current engine

The current implementation already contains the conceptual prototype of a reaction backend.

- `src/atom.h` represents symbols, variables, grounded values, and expressions as arena-allocated `Atom` nodes. Expression children are pointer arrays; structural hashes and hash-consing are available.
- `src/rhocalc_core.c` flattens parallel components, assigns alpha-invariant canonical keys, extracts send and receive endpoints, sorts endpoints by channel key, and applies COMM substitution.
- The sequential `RhoMachine` rebuilds its component and endpoint index from the residual process each round.
- The threaded strict-core path publishes endpoints into per-channel buckets and produces a residual corresponding to some legal reduction path.
- The cost profile constructs compatible waves and atomically claims endpoint and funding occurrences before commit. Causal receipts observe the same resource consumption rather than changing it.
- Canonical and rotating schedulers, exact successor frontiers, bounded execution status, differential tests, and rho-specific benchmarks already provide reference surfaces for a device backend.

The GPU compiler should preserve this semantic factoring while replacing heap strings, pointer chasing, and whole-residual reindexing on its hot path.

3 Canonical device reaction IR

3.1 Host canonicalization

Parsing, profile validation, symbol interning, and the first canonicalization remain host responsibilities in the initial backend. The host lowers bound variables to a stable locally nameless form, flattens top-level parallel composition, orients the name equation $@(*x) = x$, and interns every canonical quoted name.

Invariant 3.1 (Collision-safe identity)**.** *A* NameId *is never accepted from a hash alone. The host maintains a canonical byte representation and resolves collisions by equality checking. The device may use a compact 64- or 128-bit identifier only after this table has certified the mapping.*

3.2 Immutable term DAG

The initial ABI uses immutable nodes and contiguous child storage.

Buffer	Element type	Meaning
`nodeTag`	`u8`	nil, par, send, receive, quote, drop, value
`nodeAux`	`u64`	symbol, variable, value, or profile payload
`childOffset`	`u32/u64`	first child in contiguous child buffer
`childCount`	`u32`	arity
`children`	NodeId	immutable DAG edges
`nameOfQuote`	NameId	certified canonical name for quoted nodes

Widths are selected at compile time from a checked capacity header. A graph that exceeds the selected width is rejected or recompiled with the wider ABI; wraparound is forbidden.

3.3 Occurrence layer

Terms and occurrences are different objects. Equal terms may occur multiple times and must remain separately consumable.

Field	Type	Meaning
`componentId`	CompId	unique top-level occurrence
`componentRoot`	NodeId	immutable term DAG root
`generation`	`u32/u64`	stale-reference protection
`alive`	bitset	occurrence is present in current residual
`endpointKind`	bit	send or receive
`channelId`	NameId	canonical channel identity
`bodyOrPayload`	NodeId	send payload or receive continuation
`binderMeta`	packed word	substitution depth and profile metadata

The endpoint table is structure-of-arrays. A radix sort orders it first by channel, then by endpoint kind and component identifier; alternatively, a persistent channel index populates it directly. Segment offsets give a CSR-like channel directory.

Invariant 3.2 (Multiplicity)**.** *Every live send and receive occurrence has exactly one endpoint record, and every endpoint record refers to exactly one live component generation.*

4 Kernel pipeline

The reference device round consists of six stages.

K0: dirty-component classification. Classify newly produced components, emit endpoint records, and identify non-endpoint residuals. The first implementation may classify the entire residual; the production path updates only dirty generations.

K1: channel grouping. Sort endpoint records or insert them into a persistent segmented channel table. Produce send and receive ranges for every live channel.

K2: reaction planning. Apply the selected execution contract. A plan names exact send and receive occurrences, the channel, continuation, payload, and any funding occurrences. Planning does not mutate the residual.

K3: occurrence claim. Atomically claim every occurrence in a plan. Claims use component identity and generation, not term equality. Multi-resource plans acquire claims in a canonical order. A partial claim is rolled back; it is never exposed as a committed reaction.

K4: substitution and emission. Perform capture-avoiding substitution in the receive body and normalize the result according to the chosen rho profile. Immutable subterms are shared. New nodes are allocated through prefix-summed per-plan sizes or a checked device arena. Allocation failure is an explicit status.

K5: commit and compaction. Consume claimed occurrences, append generated components, compact dead records, update dirty channel segments, and optionally emit a reduction certificate. The result is a valid residual even if execution stops here.

Acceptance gate 4.1 (Single-round parity)**.** *For every strict-core fixture in the admitted differential corpus, decoding the device successor set shall equal the canonical CPU successor set as a set of canonical residuals. Multiplicity-sensitive tests shall additionally agree on occurrence consumption.*

5 Execution contracts must remain separate

5.1 Canonical one-path mode

This mode selects exactly the reaction chosen by the canonical CPU scheduler. It is the primary debugging oracle and may intentionally expose limited GPU parallelism. Parallel work occurs within indexing, substitution, and residual construction, not by changing the selected path.

5.2 Rotating one-path mode

The rotating scheduler preserves the current turn state and selection rule. The device result must serialize to the same scheduled path. A different legal rho path is not sufficient for scheduler parity.

5.3 Exact frontier mode

All distinct enabled pairings are enumerated and canonical duplicate residuals are collapsed exactly as in the reference engine. Search-budget or memory exhaustion is returned explicitly; it must not be reported as quiescence or as a complete frontier.

5.4 Serializable wave mode

A wave may commit several reactions only when their claimed occurrence sets are disjoint. If alternative pairing can change an observable residual, the wave planner must either enumerate the alternatives or fall back to a one-path contract. Every committed wave emits an ordering witness or passes a serializability check showing that it corresponds to a legal sequence of ordinary COMM steps.

Requirement 5.1 (No semantic performance flag)**.** *Thread count, block size, and device selection may change performance but shall not silently switch among canonical, rotating, frontier, and wave semantics.*

6 Cost-profile extension

The cost profile extends each reaction plan with its exact funding cover. Purse and signature occurrences enter the same occurrence-claim protocol as the communication endpoints.

Invariant 6.1 (Atomic funded commit)**.** *A cost reaction commits iff it owns the selected receive, selected send, and every funding occurrence in its certified cover. No residual exposes a state in which endpoints were consumed but funding was only partly acquired.*

Funding-cover selection, signature multiplication, event identifiers, producer causes, and ordered receipt material must agree with the CPU cost profile. State-only and receipt-observed device executions shall consume the same occurrences.

The first GPU milestone may exclude the cost profile, but its IR must preserve component identity and generation from the start; otherwise atomic funding cannot be added without redesign.

7 Static reaction graphs for neural computation

7.1 Compilation model

An event-driven neural network is compiled as a mostly static reaction graph: channels and handlers are immutable; spikes, traces, signals, and credit ownership are dynamic messages. A batch dimension represents independent networks, episodes, particles, or environments. Batching independent graphs is the principal source of occupancy when one graph has a sparse frontier.

The compiler emits:

- a stable table of neuron, synapse, and resource channel IDs;
- immutable handler continuations;
- per-instance dynamic occurrence buffers;
- optional fixed-capacity queues for bounded-state models; and
- observation kernels deriving outputs and weights from messages.

7.2 Conserved causal-credit fast path

The conserved-credit learner has a particularly regular image. A receipt slot, signal slot, and credit slot each occupy one finite channel state. Unary actions are one-token replacements. Redemption consumes three state tokens and publishes their three successors through three ordinary COMM steps. Derived synaptic efficacy is

$$w_k = b + q\,\#\{i \mid \mathrm{owner}(i) = k\},$$

so no dense gradient tensor or optimizer state is required.

The general backend shall execute the literal compiled rho protocol. An optional fused redemption kernel may replace the three administrative steps only after a refinement proof establishes the same residual, resource consumption, failure behavior, and observable trace under the admitted scheduler.

Invariant 7.1 (No hidden learning channel)**.** *The device fast path may cache counts or derived weights, but the authoritative learning state remains the finite receipt, signal, and credit-resource image. A cache discrepancy is a correctness failure, not an alternative optimizer.*

7.3 What bypassing PyTorch means

The required runtime consists of a native host compiler, a CUDA/HIP-style device backend, and a small C ABI. Python bindings may be added for experimentation but are not on the required execution path. There is no autograd, optimizer step, tensor module graph, or framework allocator in the steady-state loop. Dense numerical kernels may still be called where a process payload explicitly denotes a matrix operation; that is an ordinary grounded operation, not the definition of learning.

8 Correctness and certification

Let $\mathcal{C}$ compile a well-formed rho process to device state and let $\mathcal{D}$ decode a device residual to a canonical rho process. The proof and testing programme has five layers.

1. **Representation.** $\mathcal{D}(\mathcal{C}(P)) = \mathrm{canon}(P)$, including occurrence multiplicity and quoted-name identity.
2. **Single-plan soundness.** Every committed device plan decodes to one legal rho COMM step, or to the declared finite administrative sequence of a proved fused primitive.
3. **Frontier completeness.** Exact mode contains every CPU successor and no additional successor.
4. **Wave serializability.** Every wave decodes to some sequence of legal single steps with the same residual and receipt observation.
5. **Run status.** Quiescent, reduction-limit exhausted, search-budget exhausted, allocation failure, and validation failure remain distinct results.

A compact reaction certificate records profile, scheduler contract, input residual digest, selected occurrence IDs and generations, channel IDs, funding IDs, emitted roots, and output residual digest. A CPU checker replays the certificate against canonical semantics. The Lean refinement target is a decoder theorem connecting checked certificates to the established `Reduces` relation.

Acceptance gate 8.1 (Fail-closed differential gate)**.** *No optimized kernel is admitted by throughput alone. It must pass canonical single-step differential tests, scheduler-specific run tests, multiplicity tests, malformed-state rejection, exhaustion-status tests, and randomized small-process frontier comparison.*

9 Runtime API

The minimal native interface is intentionally small.

```
rf_context_create(device, options, *ctx)
rf_compile_rho(ctx, canonical_bytes, profile, *program)
rf_instantiate(program, batch_size, initial_messages, *state)
```

```
rf_step(state, execution_contract, budget, *result)
rf_run(state, execution_contract, limits, *result)
rf_decode_residual(result, instance, *canonical_bytes)
rf_export_certificate(result, *bytes)
rf_observe(result, observer_id, *buffer)
rf_destroy(handle)
```

Every call returns a typed status. Program objects are immutable and shareable; state objects own dynamic occurrences. Host/device transfer is explicit. Observers cannot mutate the reaction state.

10 Milestones and acceptance gates

Milestone	Deliverable	Admission gate
M0	Versioned flat IR and CPU decoder	Canonical round trip; multiplicity
M1	GPU endpoint extraction and channel join	Exact endpoint parity
M2	Strict-core canonical one-step backend	Successor differential corpus
M3	Rotating and exact-frontier modes	Scheduler/frontier parity; exhaustion
M4	Incremental dirty-channel index and compaction	Same gates plus speed/memory evidence
M5	Cost-profile atomic claims and receipts	Cost differential and causal replay
M6	Static neural graph compiler	Event-trace parity with CPU rho
M7	Conserved-credit device fast path	Literal-vs-fused refinement and task parity
M8	Lean certificate bridge	Checked certificate implies rho reduction

Performance gates begin only after M2 correctness. Required measurements include reactions per second, bytes moved per committed reaction, endpoint index cost, substitution cost, residual-compaction cost, active-lane fraction, batch scaling, and host/device transfer. Comparisons include the current sequential and threaded rho engines and the existing contention, fanout, pipeline, and cost-parallel workloads. A PyTorch comparison is meaningful only for a matched neural task and excludes import/startup time unless both systems are evaluated end to end.

11 Risks and required fallbacks

Irregular reflection. Dynamic quotation and process generation may produce divergent graph growth. The backend must support checked device allocation and a semantic CPU fallback at a round boundary. Falling back is preferable to truncating a process.

Small frontier under-utilization. A single sparse graph may not occupy a GPU. Batch independent graphs, keep a persistent kernel for repeated episodes, or use the CPU path. The compiler may choose a backend from static and observed frontier width, provided the semantic contract is unchanged.

Canonicalization cost. String canonical keys are unsuitable as the device hot representation. Verified name interning, structural hashes with collision checks, and incremental dirty-channel maintenance are required. Host canonical bytes remain the audit representation.

Nondeterministic atomics. Atomic arrival order must not define canonical or rotating semantics. Those modes plan deterministically before claim. A throughput wave may use atomics only within its explicitly weaker serializable-path contract.

Fused-kernel semantic drift. No neural or cost fast path is accepted because it is "obviously the same." Literal rho execution is the reference, and the fused path requires residual, failure, resource, and observation equivalence.

12 Non-goals

The first backend does not attempt to compile arbitrary grounded C functions to device code, guarantee speedup for every process, replace the host parser, or treat floating-point nondeterminism as rho nondeterminism. It does not define a new neural-network formalism. It compiles a subset of already declared grounded operations and the rho communication substrate, with explicit rejection or host fallback outside that subset.

13 Conclusion

The formatting is the optimization. Once reflective terms are separated from occurrences, canonical names from heap strings, and channel matching from recursive traversal, rho execution becomes a sequence of familiar data- parallel operations: classify, group, plan, claim, rewrite, and compact. The same representation makes a process-native neural engine plausible. Static handlers describe the graph; messages describe activity and causal learning; and conserved credit replaces an optimizer state with explicit finite resources.

The design remains rho rather than merely rho-inspired because every device transition is checked against the existing semantics. That constraint is also the engineering advantage: the CPU engine, differential corpus, causal receipts, and Lean reduction relation together provide unusually strong oracles for building an aggressive GPU backend without guessing what the program meant.